Practicals - Steps Summary

Each practical: hardware, connections (in the box), steps, output.

Practical 1 - LED blinking patterns

Hardware

Breadboard, 4 LEDs, resistors, Raspberry Pi 4, jumper wires.

Connections

```
LED 1 -> Pin 7
LED 2 -> Pin 11
LED 3 -> Pin 13
LED 4 -> Pin 15
(each LED through a resistor to GND)
```

Steps

1. Fix the 4 LEDs on the breadboard, each with a resistor to GND, and connect them to the pins in the box.
2. Open Python (IDLE or Thonny) on the Pi and make a new file.
3. Type the program: it imports `RPi.GPIO as x` and `time`, uses `x.setmode(x.BOARD)`, sets pins 7, 11, 13, 15 as OUT, then repeats forever.
4. Pattern 1: all 4 LEDs ON and OFF, 3 times. Pattern 2: first two LEDs, then last two LEDs, 3 times. Pattern 3: LEDs switch on one by one (1000, 1100, 1110, 1111) then switch off in reverse, 3 times. Every step waits 0.5 seconds.
5. Save and run the file.

Output

LEDs blink in the three patterns, one after another.

Practical 2 - Telegram controls Raspberry Pi GPIO

Hardware

Breadboard, 4 LEDs (green, yellow, red, blue), resistors, Raspberry Pi 4, jumper wires, a phone with Telegram.

Connections

```
Green  +  -> Pin 3  (GPIO 2)
Yellow +  -> Pin 5  (GPIO 3)
Red    +  -> Pin 7  (GPIO 4)
Blue   +  -> Pin 11 (GPIO 17)
Negative of all four -> Pin 9 (GND)
```

Steps

1. Open Terminal on the Pi and type `sudo apt-get install idle`, then `sudo pip install telepot`.
2. On your phone, install Telegram and log in with your phone number and the verification code.
3. In Telegram search `BotFather`
4. Click Start and send `/newbot`. Give a name (example: rpi4control_ab), then a username ending with bot (example: rpi4control_abbot).
5. BotFather replies with a token. Save it and keep it private.

6. Connect the LEDs as shown in the box.
7. Open IDLE, type the script, and replace `"YOUR TOKEN"` with your token.
8. Open Terminal and run `python test.py`. Wait for `Up and Running...`.
9. In Telegram send messages to your bot, for example `blue on`, `all on`, `red off`, `all off`.

Output

The bot replies (example: On blue light(s)) and the LEDs switch on or off. The script looks for the words on / off plus a colour or all.

Practical 3 - TM1637 4-digit display shows time

Hardware

Raspberry Pi, TM1637 4-digit 7-segment display, female-to-female jumper wires.

Connections

```
GND -> Pin 14
VCC -> Pin 4  (5V)
DIO -> Pin 18 (GPIO 24)
CLK -> Pin 16 (GPIO 23)
```

Steps

1. Connect the display as shown in the box.
2. Open Terminal and type `pip3 show tm1637`, then `pip3 install python-tm1637`.
3. Open Python and type the code: `tm1637.TM1637(clk=23, dio=24)`, then a loop that reads the current hour and minute and shows them with `Display.numbers(hour, minute, sep=colon)`.
4. The colon is switched on and off each second, and the loop waits 1 second.
5. Run the program.

Output

The display shows the current time with a blinking colon.

Practical 4 - GPS module

Hardware

Raspberry Pi (the diagram says Pi 3) and a GPS module. Your PDF has only the circuit diagram and code, no aim or steps.

Connections

```
Pin 6  (GND) -> GPS GND
Pin 8  (TXD) -> GPS RXD
Pin 10 (RXD) -> GPS TXD
Pin 4  (5V)  -> GPS VCC
```

Steps

1. Connect the GPS module as shown in the box (TX and RX are crossed).
2. If serial is not enabled: Open Terminal, type `sudo raspi-config`, go to Interface Options, Serial Port, login shell No, hardware Yes, then `sudo reboot` (same as in practicals 11 and 12).
3. Open Python and type the code: it opens `serial.Serial("/dev/serial0", baudrate=9600, timeout=1)` and keeps reading and printing lines.
4. Run the program.

Output

GPS data lines keep printing on the screen.

Practical 5 - Raspberry Pi oscilloscope (ADS1115)

Hardware

Raspberry Pi A/B/B+, ADS1115 ADC, breadboard, jumper wires.

Connections

```
VDD -> Pin 17 (3.3V)
GND -> Pin 9
SCL -> Pin 5  (GPIO 3)
SDA -> Pin 3  (GPIO 2)
```

Steps

1. Connect the ADS1115 through the breadboard as shown in the box.
2. Open Terminal, type `sudo raspi-config`, go to Interface Options > I2C > Enable.
3. Type `sudo apt-get update` then `sudo apt-get upgrade`.
4. Type `cd ~` then `sudo apt-get install build-essential python3-dev python3-smbus git`.
5. Type `git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git`, then `cd Adafruit_Python_ADS1x15`, then `sudo python3 setup.py install`.
6. Test it: `cd examples` then `python3 simpletest.py`. A table of the 4 channel values appears. (Option: the PDF also has a test script for channel 0, installed with `pip3 install adafruit-circuitpython-ads1x15`, which prints raw value and voltage every 0.5 seconds.)
7. Type `sudo apt-get install python3-matplotlib`.
8. Type `sudo apt-get install python3-pip` then `sudo pip3 install drawnow`.
9. Type `git clone https://github.com/tyitraspberrypi/Adafruit_Python_ADS1x1z_graph.git` and run the graph code from it.

Output

A live graph titled Osciloscope shows Channel 0.

Practical 6 - 16x2 I2C LCD

Hardware

Raspberry Pi 400, LCD 1602 with I2C module (PCF8574 backpack), 4 female-to-female jumper wires.

Connections

```
GND -> Pin 6
VCC -> Pin 4 (5V)
SDA -> Pin 3 (GPIO 2)
SCL -> Pin 5 (GPIO 3)
```

Steps

1. Connect the LCD as shown in the box.
2. Open Terminal, type `sudo raspi-config`, go to Interface Options > I2C > Enable, then reboot the Pi.

3. Type `sudo apt update`, then `sudo apt install i2c-tools python3-smbus python3-pip`, then `pip3 install RPLCD`. (On Bookworm add `--break-system-packages` or use a virtual environment.)
4. Type `i2cdetect -y 1` and note the address (0x27 in your PDF, sometimes 0x3F).
5. Save `lcd_test.py`: it uses `CharLCD('PCF8574', 0x27, cols=16, rows=2)`, writes `Hello from Pi 400!`, waits 3 seconds and clears. Run `python3 lcd_test.py`.
6. Optional: save `custom_message.py`. It asks for line 1 and line 2 (max 16 characters each), shows them for 5 seconds and clears.

Output

The message appears on the LCD.

Practical 7 - RFID read, write and erase

Hardware

Raspberry Pi 4, RFID module (MFRC522), RFID tag/card, jumper wires. The module runs on 3.3V.

Connections

```
Vcc  -> 3.3V (Pin 1)
Gnd  -> Gnd (Pin 6)
SDA  -> CS0 SPI0 (Pin 24)
SCK  -> SCLK (Pin 23)
MOSI -> MOSI 0 (Pin 19)
MISO -> MISO 0 (Pin 21)
```

Steps

1. Connect the RFID module as shown in the box.
2. Open Terminal, type `sudo raspi-config` and enable SPI. Then type `lsmod | grep spi` to check.
3. Type `sudo apt-get update`, `sudo apt-get upgrade`, then `sudo pip3 install mfrc522`.
4. Open IDLE or Thonny and make three programs (use normal straight quotes when typing).
5. write.py: asks `Enter New data:`, prints `Place tag`, writes the text to the tag.
6. read.py: prints `Place the RFID tag near the reader...`, then shows Tag ID and the data stored.
7. erase: writes 16 blank spaces to the tag. It prints `SUCCESS: Tag data wiped successfully!`.

Output

Write: Data written. Read: id = a long number, Data = IOT Practical. Erase: Tag data wiped successfully.

Practical 11 - Fingerprint sensor + I2C LCD

Hardware

Raspberry Pi 4 Model B, R307/R305 fingerprint sensor, 16x2 LCD with PCF8574 (HW-61) I2C backpack.

Connections

```
LCD:
GND -> Pin 6
VCC -> Pin 2 (5V)
SDA -> Pin 3 (GPIO 2)
SCL -> Pin 5 (GPIO 3)

Fingerprint sensor:
Red (VCC)        -> Pin 4 (5V)
Black (GND)      -> Pin 9 or 14
Yellow/Green TXD -> Pin 10 (GPIO 15 RXD)
White/Blue RXD   -> Pin 8  (GPIO 14 TXD)
```

Steps

1. Power off the Pi and wire everything as shown in the box.
2. Power on. Open Terminal, type `sudo raspi-config`, go to Interface Options. Enable I2C. In Serial Port choose No for login shell and Yes for hardware serial. Then type `sudo reboot`.
3. Type `sudo apt-get update && sudo apt-get install -y i2c-tools python3-pip python3-serial`.
4. Type `pip3 install RPLCD adafruit-circuitpython-fingerprint smbus2`.
5. Type `i2cdetect -y 1` to check the LCD address (0x27 or 0x3f).
6. Run `python3 enroll.py`. Enter a slot ID (1-127), place your finger, remove it, place the same finger again.
7. Run `python3 main_verify.py`. The LCD shows System Ready / Place Finger. Place a finger.

Output

Enrolled finger: LCD shows FINGER MATCHED and the ID. Other finger: FINGERPRINT UNMATCHED, then Access Denied. If LCD text is not visible, turn the blue screw on the back of the HW-61.

Practical 12 - Fingerprint door lock with relay

Hardware

Raspberry Pi 4, relay module (1-channel), R307/R305 fingerprint sensor, female-to-male jumper wires.

Connections

```
Relay:
VCC -> Pin 2 (5V)
GND -> Pin 9
IN  -> Pin 11 (GPIO 17)

Fingerprint sensor:
VCC -> Pin 2 (5V)
GND -> Pin 6
TX  -> Pin 10 (GPIO 15 RXD)
RX  -> Pin 8  (GPIO 14 TXD)
```

Steps

1. Connect the relay and sensor as shown in the box.
2. Open Terminal and type `sudo apt update` then `sudo apt upgrade -y`.
3. Type `sudo apt install -y python3 python3-pip python3-serial`.
4. Type `pip3 install adafruit-circuitpython-fingerprint`.
5. Type `sudo raspi-config`, go to Interface Options > Serial Port: login shell No, hardware Yes. Type `sudo reboot`. After reboot type `ls -l /dev/serial0`.
6. If the OS blocks pip: `python3 -m venv fingerprint-env`, then `source fingerprint-env/bin/activate`, then `pip install pyserial adafruit-circuitpython-fingerprint RPi.GPIO`.
7. Run `python3 enroll.py`. Enter ID (1-127), place finger, remove it, place it again. It prints Fingerprint enrolled successfully and the Stored ID.
8. Run `python3 door_lock.py` (in Thonny or IDLE). It prints Fingerprint Door Lock System, Place Finger. Place a finger.

Output

Enrolled finger: Access Granted, relay ON for 5 seconds (Door Unlocked), then Door Locked. Other finger: Access Denied. The relay is active-LOW (LOW = ON).

Practical 13 - MQTT publisher and subscriber (LED)

Hardware

Raspberry Pi A/B/B+, LED, female-to-male jumper wires, a phone with the MQTT Dashboard app (Play Store).

Connections

```
LED / VCC -> Pin 12 (GPIO 18)
GND         -> Pin 6
```

Steps

1. Connect the LED as shown in the box.
2. On the phone: open the Play Store and install MQTT Dashboard.
3. Open the app, go to Broker and add details: Client ID `led001`, Server `broker.emqx.io`, Port `1883`. Leave username and password empty and tick to save.
4. Go to Publisher, tap +, choose Button. Make two buttons: name `led ON`, topic `led`, value `ON`; and name `led off`, topic `led`, value `off`.
5. On the Pi, open Terminal and type `python3 -m venv myenv`, then `source myenv/bin/activate`, then `pip install paho-mqtt`.
6. Run `python3 test.py`. It prints Waiting for MQTT messages, Connected with result code: Success, Subscribed to topic: led.
7. Press the ON and off buttons in the app.

Output

Terminal prints Received: on / off and LED ON / LED OFF, and the LED switches.

Practical 14 - Flask web dashboard to control a relay

Hardware

Raspberry Pi A/B/B+, relay module, female-to-male jumper wires.

Connections

```
Relay VCC -> Pin 2 (VCC)
Relay GND -> Pin 6 (GND)
CH 1 IN 1 -> Pin 12 (GPIO 18)
```

Steps

1. Connect the relay as shown in the box. (Your PDF code says RELAY = 17 but the table says GPIO 18. Wire the pin that matches the code you run.)
2. Open Terminal and type `sudo apt update`, then `sudo apt install python3-flask`.
3. Type `python3 -c "import flask ; print ('Flask OK')"` to check.
4. Type `nano relay.py`, paste the code, save.
5. Type `python3 relay.py`.
6. Open another terminal and type `hostname -I` to get the Pi IP address.
7. On a phone or PC browser open `IP-address:5000`. Press ON and OFF.

Output

The page Raspberry Pi Relay shows Status ON/OFF with two buttons. The relay switches and the terminal logs GET /on and GET /off.

Practical 17 - Node-RED push button + LED

Hardware

Raspberry Pi 400, power adapter, HDMI to micro HDMI cable, mouse, monitor, push button, resistor, LED, jumper wires.

Connections

```
Pin numbers are not written in your notebook.
Push button -> a GPIO input pin (you choose)
LED + resistor -> a GPIO output pin (you choose)
LED: long leg (anode +) to the GPIO via resistor,
     short leg (cathode -) to GND
```

Steps

1. Connect the push button and LED as shown in the box.
2. Open Terminal and install Node-RED with the command written in your notebook: `bash <(curl -sL https://github.com/node-red/linux-installer/releases/latest/download/install-update-nodered-deb)`. (If it fails, use the same command from the official Node-RED Raspberry Pi page.)
3. Type `node-red-start` and open the URL shown in the terminal.
4. Build the flow: GPIO input, Switch (1), Switch (2), Switch (3), Change (1), Change (2), GPIO output.
5. GPIO input: select the pin of the push button. GPIO output: select the pin of the LED.
6. Switch (1): property global.state, rule 1 == 0, rule 2 == 1.
7. Switch (2): property msg.payload, rule == 1.
8. Switch (3): property global.state, rule 1 == 0, rule 2 == (the notebook looks like 4, check it).

9. Change (1): set global.state = 1 and msg.payload = 1. Change (2): set global.state = 0 and msg.payload = 0.
10. Deploy the flow and press the button.
11. Other commands: `node-red-stop`, `node-red-restart`, `node-red-reload`.

Output

The LED toggles ON/OFF from the physical button (and from the web dashboard). Button release is ignored because the payload is not 1.

Practical 18 - CPU%, memory% and CPU temperature

Hardware

Raspberry Pi A/B/B+ (Raspbian OS).

Connections

```
No connections (software only)
```

Steps

1. Open Terminal and type `sudo apt update`.
2. Type `sudo apt install python3-psutil`.
3. Save the code as `test.py`: it uses psutil for CPU % and memory %, and reads the temperature from `/sys/class/thermal/thermal_zone0/temp`, then prints every second.
4. Type `python3 test.py`.

Output

CPU Usage, Memory Usage and CPU Temperature print every second.

Practical 19 - Access control (RFID + fingerprint + LCD + relay)

Hardware

Not listed in the PDF. From the code: Raspberry Pi, RFID reader, fingerprint sensor, 16x2 I2C LCD, relay, LED.

Connections

```
No connection table in the PDF.
From the code: Relay -> GPIO 23, LED -> GPIO 21
I2C LCD address: 0x27 (use 0x3F if i2cdetect shows 0x3F)
RFID, fingerprint and LCD: wire as in practicals 7 and 11
```

Steps

1. Open Terminal and type `sudo apt update`.
2. Type `sudo apt install -y python3-pip python3-serial python3-rpi.gpio i2c-tools`.
3. Type `pip3 install RPLCD adafruit-circuitpython-fingerprint mfrc522 --break-system-packages`.
4. Programme 01 (RFID): run it, place your card, note the UID it prints.
5. Programme 02 (fingerprint): run it, enter an ID (1-127), place finger, remove, place again to enroll.
6. Programme 03 (main): put your card UIDs in the WHITELIST list, then run it.

7. The LCD shows ACCESS CONTROL / SYSTEM READY, then Scan RFID or Fingerprint. Scan a card or finger.

Output

Allowed: ACCESS GRANTED, LED on, relay on for 5 seconds. Not allowed: ACCESS DENIED for 2 seconds. Every attempt is saved in access_log.csv (Time, Method, Result). Ctrl+C stops it.